

Agenda

- Tailwind CDN
- Colors
- Sizing
- Spacing
- Positions
- Breakpoints
- flex
- grid
- Directive
- darkmode
- predefined component library
- Javascript Introduction
- Language Fundamentals

Introduction

- Prerequisites for tailwind is Html and Css
- No framework is best until we don't know the requirements
- the tailwind is built in Javascript
- It works on utility first principle i.e for every css property it has got a class

Colors

- Tailwind CSS includes a vast, beautiful color palette out of the box, carefully crafted by expert designers and suitable for a wide range of different design styles.
- Every color in the default palette includes 11 steps, with 50 being the lightest, and 950 being the darkest:
- The entire color palette is available across all color related utilities, including things like background color, border color, fill, caret color, and many more.
- Use color utilities like `bg-white`, `border-pink-300`, and `text-gray-950` to set the different color properties of elements in your design

```
<div class="bg-sky-900"></div>
/* bg-* Sets the background color of an element */
<p class="text-gray-700">Hello</p>
/*text-* Sets the text color of an element */
```

- however the 11 shades are not applicable to white and black color

Sizing (Width and Height)

1. Width

- Utilities for setting the width of an element.

- Use w- utilities like w-24 and w-64 to set an element to a fixed width based on the spacing scale
- Use w-full or w- utilities like w-1/2 and w-2/5 to give an element a percentage-based width:

```
<!-- width of div consumed as 4 px -->
<div class="bg-red-500 w-1 m-2">Width and Height</div>

<!-- width of div consumed as 80 px -->
<div class="bg-red-500 w-20 m-2">Width and Height</div>

<!-- can be provided with fixed pixels -->
<div class="bg-red-500 w-[135px] m-2">Width and Height</div>

<!-- max width is set to 50 which cannot be further extended -->
<div class="bg-red-500 max-w-50 w-100 m-2">Width</div>

<!-- min width is set to 50 which cannot be further Shrunked -->
<div class="bg-red-500 min-w-50 w-20 m-2"> Width</div>

<div class="bg-red-500 w-1/2">Width and Height</div>
<div class="bg-red-500 w-2/3">Width and Height</div>
```

2. Height

- Utilities for setting the height of an element.
- Use h- utilities like h-24 and h-64 to set an element to a fixed height based on the spacing scale
- Use h-full or h- utilities like h-1/2 and h-2/5 to give an element a percentage-based height
- Use the h-screen utility to make an element span the entire height of the viewport

```
<body class="h-screen">
  <!-- width of div consumed as 4 px -->
  <div class="bg-red-500 h-10 m-2">Width and Height</div>
  <div class="bg-red-500 h-50 m-2">Width and Height</div>

  <div class="bg-red-500 max-h-50 m-2 h-200">Width and Height(max)</div>
  <div class="bg-red-500 min-h-50 m-2 h-20">Width and Height (min)</div>

  <div class="bg-red-500 h-2/5 m-2">Width and Height</div>
  <div class="bg-red-500 h-full m-2">Width and Height</div>
</body>
```

size

- if the height and width values are same then size class must be used

```
<body class="h-screen">
  <div class="bg-amber-500 w-50 h-50 m-2">Sunbeam</div>
```

```

<!-- for same height and width use can use size -->
<div class="bg-amber-500 size-50 m-2">Sunbeam</div>

<!-- min and max cannot be used with size -->
<div class="bg-amber-500 min-size-40 m-2">Sunbeam</div>

<div class="bg-amber-500 size-[79px] m-2">Sunbeam</div>

<div class="bg-amber-500 size-1/2 m-2">Sunbeam</div>
<div class="bg-amber-500 size-full m-2">Sunbeam</div>

</body>

```

Spacing (Margin and Padding)

- Utilities for controlling an element's margin.
- Use m- utilities like m-4 and m-8 to control the margin on all sides of an element
- Use mt-, mr-, mb-, and ml- utilities like ml-2 and mt-6 to control the margin on one side of an element
- Use mx- utilities like mx-4 and mx-8 to control the horizontal margin of an element
- Use my- utilities like my-4 and my-8 to control the vertical margin of an element:
- Apply the same for padding with **m** replaced by **p**
- Note that 1 rem is considered as 16 px in css however tailwind uses a standard spacing scale where 1 unit equals 0.25rem (which is 4px)

```

<h1 class="bg-red-500 text-white m-4 p-4">Margin and Padding</h1>
<h1 class="bg-red-500 text-white mx-4 px-4">Margin and Padding</h1>
<h1 class="bg-red-500 text-white my-4 py-4">Margin and Padding</h1>
<h1 class="bg-red-500 text-white ml-4 mb-8 pl-4 pb-4">Margin and Padding</h1>
<h1 class="bg-red-500 text-white mr-8 mt-16 pr-8,pt-16">Margin and Padding</h1>

```

Positions

- By default, everything has a position of static, meaning it just flows naturally down the page. Changing the position allows you to move elements around using coordinates like top, bottom, left, and right (top-0, right-4, etc.).

Tailwind Class	Plain CSS	How it Behaves	Common Use Case
static	<code>position: static;</code>	Default. Flows naturally with the page. Top/bottom/left/right settings are ignored.	Standard text, paragraphs, simple layouts.
relative	<code>position: relative;</code>	Stays in the natural flow, but acts as an anchor/parent for absolute elements.	Container for dropdowns, tooltips, or badges.

Tailwind Class	Plain CSS	How it Behaves	Common Use Case
absolute	<code>position: absolute;</code>	Breaks out of the flow. Positions itself relative to its closest non-static parent .	Close buttons (X) on modals, notification badges.
fixed	<code>position: fixed;</code>	Breaks out of the flow. Stays locked to the browser viewport . Doesn't move when scrolling.	Sticky navigation bars, chat widgets, pop-ups.
sticky	<code>position: sticky;</code>	Acts like relative until you scroll to a certain point, then locks like fixed .	Table headers, scrolling sidebars.

Responsive Design(Breakpoints)

- Using responsive utility variants to build adaptive user interfaces.
- First, make sure you've added the viewport meta tag to the of your document:
- Then to add a utility but only have it take effect at a certain breakpoint, all you need to do is prefix the utility with the breakpoint name, followed by the : character:
- There are five breakpoints by default, inspired by common device resolutions:

Breakpoint	Minimum width	CSS
sm	40rem (640px)	@media (min-width >= 40rem){...}
md	48rem (768px)	@media (min-width >= 48rem){...}
lg	64rem (1024px)	@media (min-width >= 64rem){...}
xl	80rem (1280px)	@media (min-width >= 80rem){...}
2xl	96rem (1536px)	@media (min-width >= 96rem){...}

- This works for every utility class in the framework, which means you can change literally anything at a given breakpoint — even things like letter spacing or cursor styles.

Directive (At-Rules)

- At-rules are CSS statements that instruct CSS how to behave.
- They begin with an at-sign, @ (U+0040 COMMERCIAL AT), followed by an identifier.
- They include everything from the at-keyword up to the next semicolon, ; (U+003B SEMICOLON), or the next CSS block, whichever comes first.
- At-rules are used to group and structure style rules and other at-rules, declare style information not directly associated with selected content, and manage syntactic constructs such as imports and namespaces keyword mappings.

```
/* General structure */
@identifier (RULE);
```

```
/* Example: tells browser to use UTF-8 character set */  
@charset "utf-8";
```

- Directives are custom Tailwind-specific at-rules you can use in your CSS that offer special functionality for Tailwind CSS projects.
- In tailwind few important directives are

1. @import

- Use the @import directive to inline import CSS files, including Tailwind itself:

2. @apply

- Use the @apply directive to inline any existing utility classes into your own custom CSS:

```
<head>  
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>  
  <style type="text/tailwindcss">  
    h1 {  
      @apply text-6xl  
      bg-black  
      text-white;  
    }  
  </style>  
</head>  
  
<body>  
  <h1>Header 1</h1>  
  <h1>Header 2</h1>  
  <h1>Header 3</h1>  
  <h1>Header 4</h1>  
</body>
```

JavaScript

- Invented by Brendan Eich in 1995 at Netscape Corporation for Netscape2
- It is a Scripting language for web development
- It is an interpreted language
- Used to develop server side programs (Node) as well
- It is an Object Oriented Programming Language
- Use script tag to write JS code in html page
- Loosely typed language
- It is used to dynamically modify the html pages. It has full integration with HTML/CSS
- All major browsers support and by default enabled for javascript
- ECMAScript is the official name of the language.
- ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.
- Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, 2020).

Types of javascript

1. Internal

- It is inserted into the documents by using the script tag
- script tag provides a block to write the java script programs

```
<script>  
    JS code goes here  
</script>
```

2. External

- To use the predefined programs of any javascript library.

```
<script src = "myscript.js"></script>
```

Built-in Objects

1. console

- represents the web console (terminal)
- use log method to write output on console

2. window

- used to display alert, prompt or confirmation

3. document:

- represents DOM (Document Object Model)
- Collection of objects of all the elements present inside the page

Variables

- It is a container to store the data
- To declare a variable data type MUST NOT be used in its declaration
- To create variables in JS we can use

1. var

- It is a normal variable whose value can be changed multiple times
- var is not used as it is deprecated, it is recommend to use let instead

2. let

- It is a normal variable whose value can be changed multiple times

3. const

- It is a variable whose value cannot be changed once initialized.

- modern JS prefer "let" & "const" over "var" keyword.
- Syntax: let name = initial value; const name = initial value;
- E.g.

```
let num = 100; // number
const salary = 4.5; // number
let = "test"; // string
const firstName = 'steve'; // string
let canVote = true; // Boolean
```

Data Types

- In JS, all Data Types are inferred (automatically decided by JS)
- typeof operator can be used to get the type of the variable.
- JavaScript has 8 built-in data types.
- These are divided into two main categories:

1. Primitives (which hold a single value and are immutable)
2. Objects (which are collections of properties).

1. Primitive Data Types

- Primitive values are stored directly in the location that the variable accesses.
- They are immutable (cannot be altered)

1. number

- Handles both integer and floating-point values.
- It also includes special numeric values: Infinity, -Infinity, and NaN (Not a Number)
- E.g.
 - num = 100;
 - salary = 4.5;

2. string

- Must be surrounded by single quotes (' '), double quotes (" "), or backticks (` `).
- Backticks allow you to embed variables using \${variable}
- E.g.
 - firstName = "steve";
 - lastName = 'Jobs';

3. boolean

- Used for conditional testing.
- It only has two values: true and false
- E.g.
 - canVote = true;
 - canVote = false;

4. null

- A special value that represents "nothing", "empty", or "value unknown"

5. undefined

- Means a variable has been declared, but has not yet been assigned a value.

Built-in Values

1. NaN

- Not a Number
- Is of type number
- E.g. `console.log(parseInt("test"));`

2. Infinity:

- When a number is divided by 0
- E.g. `answer = 10 / 0; // Infinity`

3. undefined:

- No value/type present

4. null:

- No object.
- is of type "object"

null vs undefined

null

- null represents an intentional absence of any object value, while undefined represents a variable that has been declared but not yet given a value.
- null is used when you want to show that a variable is currently empty on purpose.
- used to reset an object reference or clear out old data.

undefined

- A variable automatically becomes undefined when you declare it with `let` or `var` without giving it a value.
- JS returns undefined when you try to access an object property or a function argument that does not exist.